

HW8 Team Report

Random Team: Yinuo Feng, Chenyu Jiang, Ziheng Chen, Boyuan Sun, Nuanxin Jin

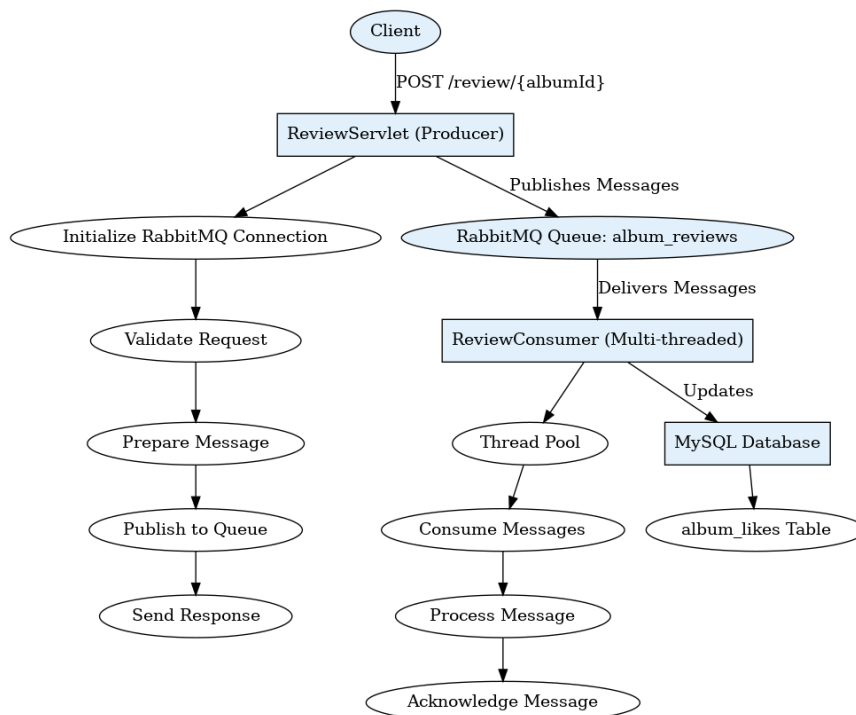
Each team member implemented their own server and conducted independent testing based on the same problem description. The following report summarizes all five designs and results.

1. Server Design Summaries

Ziheng Chen (Java + RabbitMQ + Multithreaded Consumers)

Ziheng implemented a Java-based system using RabbitMQ for asynchronous processing. The ReviewServlet validates incoming like/dislike requests and publishes JSON messages to a durable queue. A pool of consumer threads (configurable via thread pool) consumes messages and inserts them into a MySQL album_likes table. Manual acknowledgement and per-request channel creation are used to ensure delivery guarantees.

Diagram:



Yinuo Feng (Java, Threaded HTTP Client, Emphasis on Throughput)

Yinuo focused on high-concurrency client simulation with a fixed 10-30-2 configuration and used metrics like latency percentiles and total success rates. The server side follows a similar structure using RabbitMQ but with an optimized batching strategy.

Chenyu Jiang (Java + Single-threaded Consumer)

Chenyu's design follows a single-threaded RabbitMQ consumer. Her test shows that even with one thread, the system maintained a small queue size and reasonable throughput. She speculated that the Java servlet overhead might become a bottleneck in higher loads, especially compared to Go implementations.

Boyuan Sun (Java multi-threaded Client & Server + Stand alone multi-threaded Java consumer + RabbitMQ)

Client posts album -> Server inserts album info to database -> Database return album ID -> Server return the album ID to client -> client post likes using the album ID -> Server publishes likes to RabbitMQ -> Consumer grab likes from RabbitMQ and insert into database

Nuanxin Jin (Go + Three Architecture Variants)

Nuanxin tested three setups:

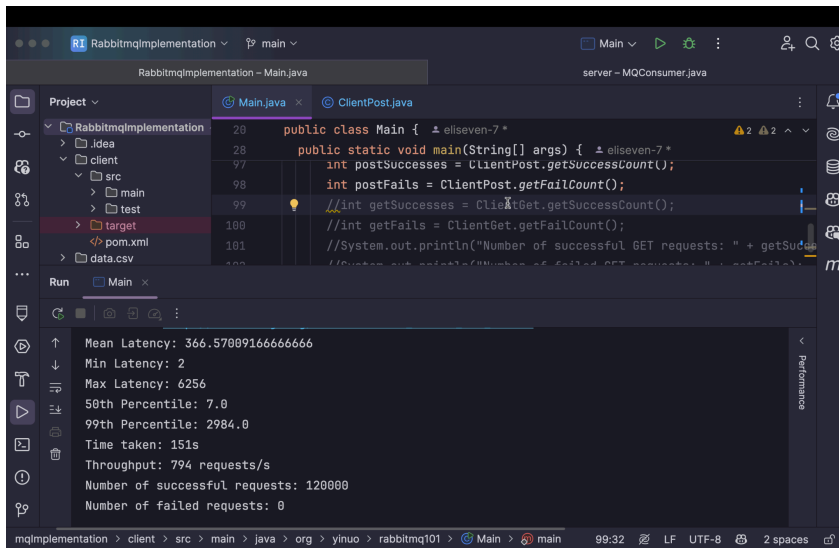
1. Single-threaded with send-back
2. Multi-threaded without send-back
3. Multi-threaded with send-back + prefetch control

She showed detailed tradeoffs, concluding that limiting prefetch to 50 increased performance and reduced queue length.

2. Performance Results and Bottlenecks Observed

Note: Comparisons are made for instance 10 30 2.

Yinuo Result:

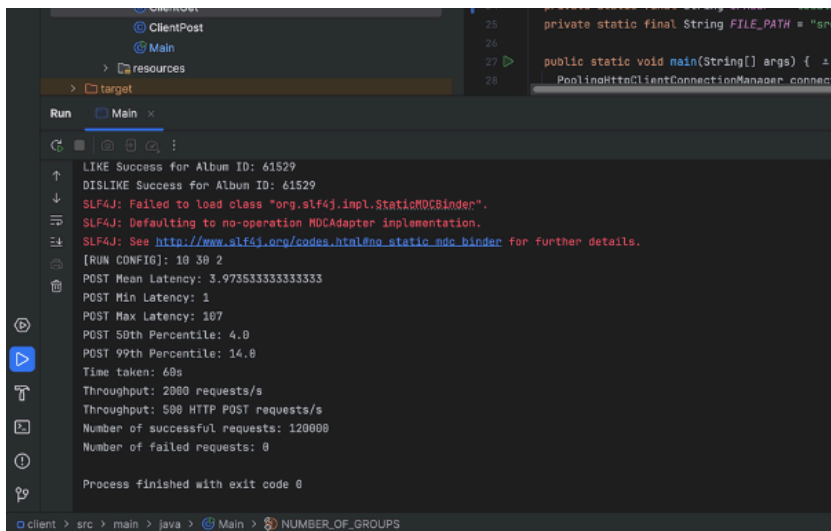


```
Mean Latency: 366.57009166666666
Min Latency: 2
Max Latency: 6256
50th Percentile: 7.0
99th Percentile: 2984.0
Time taken: 151s
Throughput: 794 requests/s
Number of successful requests: 120000
Number of failed requests: 0
```

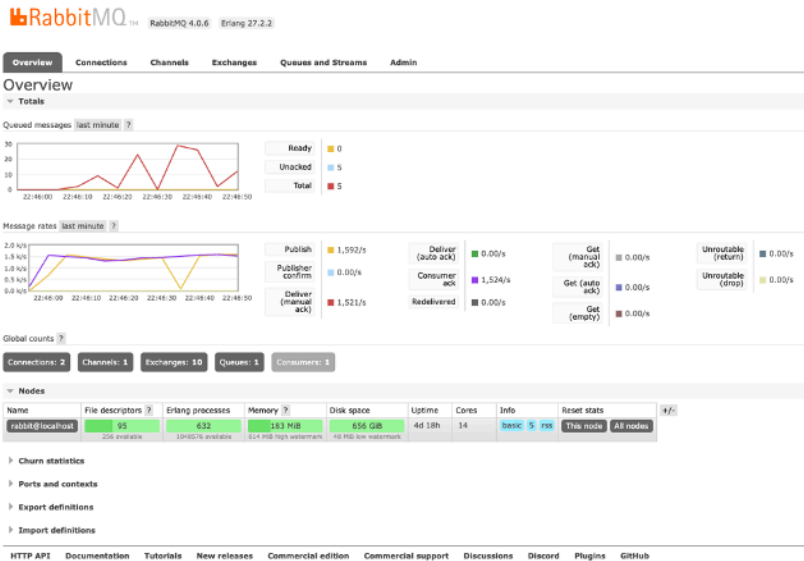
Possible Bottlenecks

1. If the server establishes a new RabbitMQ connection every time it posts a review, it will cause overhead.
2. If the consumer is single-threaded, it does not work as efficiently as multithreaded MQ consumers.

Chenyu Result:



```
LIKE Success for Album ID: 61529
DISLIKE Success for Album ID: 61529
SLF4J: Failed to load class "org.slf4j.impl.StaticMDCBinder".
SLF4J: Defaulting to no-operation MDCAdapter implementation.
SLF4J: See http://www.slf4j.org/codes.html#no-static-mdc-binder for further details.
[RUN CONFIG]: 10 30 2
POST Mean Latency: 3.973533333333333
POST Min Latency: 1
POST Max Latency: 107
POST 50th Percentile: 4.0
POST 99th Percentile: 14.0
Time taken: 60s
Throughput: 2000 requests/s
Throughput: 500 HTTP POST requests/s
Number of successful requests: 120000
Number of failed requests: 0
Process finished with exit code 0
```



The queue size is quite small and the throughput is satisfactory while I am using a single thread for RabbitMQ consumer. One possible bottleneck here could be the using Java servlet, which might limit the performance compared to the implementations using Go, as I have witnessed higher throughput from other teams.

Nuanxin Result:

Client: Main.go – handling http requests and passing the data to the broker.

Broker: rabbitmq

Server: database – receiving messages from the broker and write them into database

1. Single thread with send back:

Post album:

Client(main.go) -----postAlbum (album info) -----> Broker

Broker -----postAlbum (album info) -----> Server(db.go)

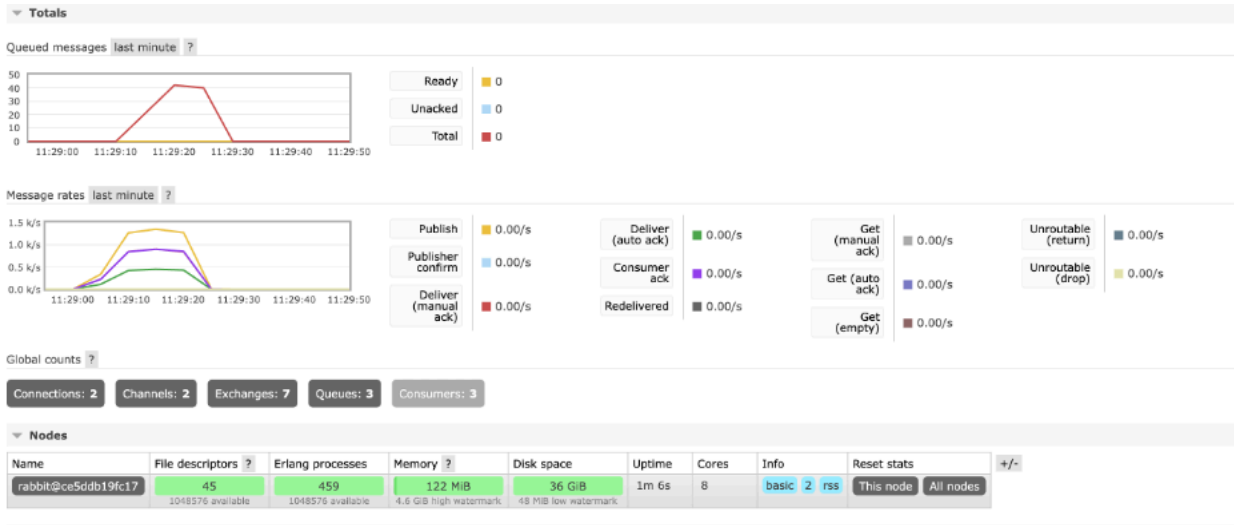
Client(db.go) ----- resAlbumQueue (album ID) ---->Broker

Broker ----- resAlbumQueue (album ID)----->Server(main.go)

Post like:

Client(main.go)-----preferredQueue (like infor) -----> Broker

Broker-----preferredQueue (like infor) -----> Server(db.go)



```
Time taken: 27s
Throughput: 740 requests/s

Process finished with exit code 0
```

The latency is considerable—under the 10×10×2 group configuration, the throughput reaches only 740. Based on observations, the primary bottleneck appears to lie in the db.go file, particularly in the way goroutines are handled. In addition to increased latency, this issue may also negatively impact the database's performance, especially when handling concurrent write operations.

2. Multi-thread without send back:

Post album:

Client(main.go) -----postAlbum (album info) -----> Broker

Broker -----postAlbum (album info) -----> Server(db.go)

Post like:

Client(main.go)-----preferredQueue (like infor) -----> Broker

Broker-----preferredQueue (like infor) -----> Server(db.go)

The latency significantly improves.

10*30*2:

```
Time taken: 58s
Throughput: 1034 requests/s
```

Overview

▼ Totals

Queued messages last ten minutes ?



Ready	60,303
Unacked	0
Total	60,303

Message rates last ten minutes ?



Publish	0.00/s	Deliver (auto ack)	0.00/s	Get (manual ack)	0.00/s	Unroutable (return)	0.00/s
Publisher confirm	0.00/s	Consumer ack	0.00/s	Get (auto ack)	0.00/s	Unroutable (drop)	0.00/s
Deliver (manual ack)	0.00/s	Redelivered	0.00/s	Get (empty)	0.00/s		

3. Multi-thread with send back:

Post album:

Client(main.go) -----postAlbum (album info) -----> Broker

Broker -----postAlbum (album info) -----> Server(db.go)

Client(db.go) ----- resAlbumQueue (album ID) ---->Broker

Broker ----- resAlbumQueue (album ID)----->Server(main.go)

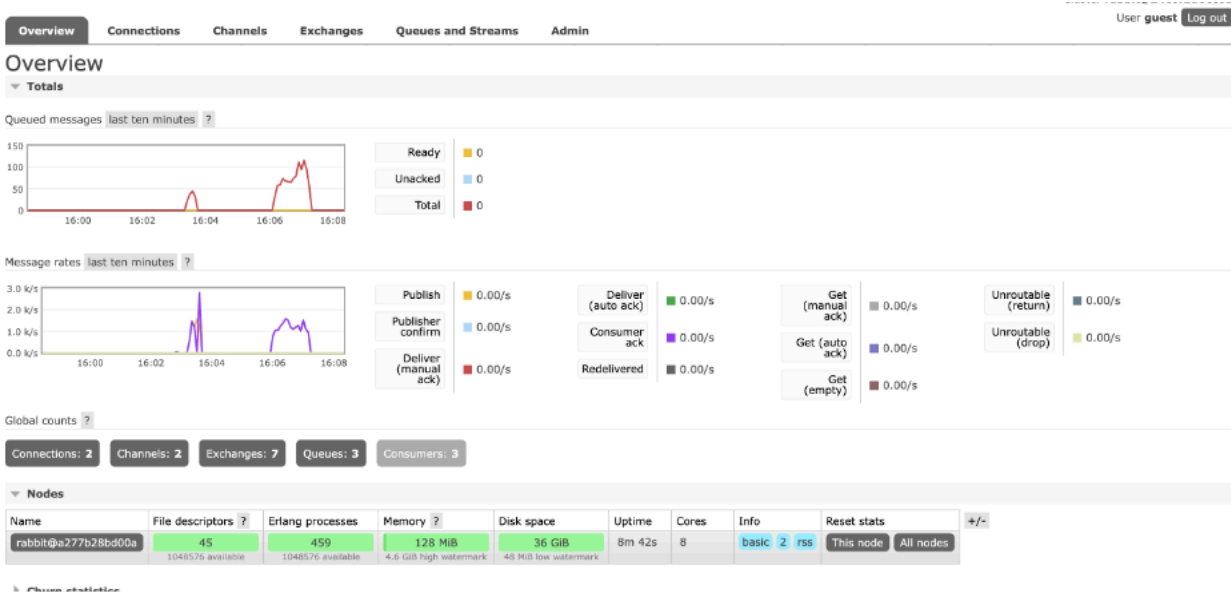
Post like:

Client(main.go)-----preferredQueue (like infor) -----> Broker

Broker-----preferredQueue (like infor) -----> Server(db.go)

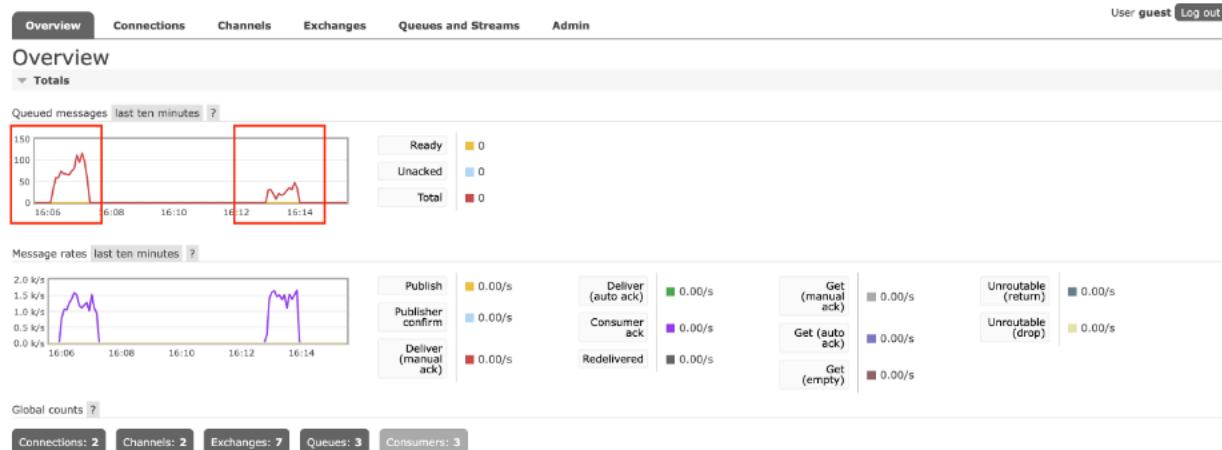
With unlimited prefetch count:

```
Time taken: 72s
Throughput: 833 requests/s
```



With 50 prefetch count:

```
Time taken: 61s
Throughput: 983 requests/s
```



With a prefetch value of 50, performance increased by 18%, and the queue size was reduced by half. This improvement may be due to the server not being overwhelmed when releasing messages.

Additionally, compared to the single-threaded approach with send-back, latency improved by 32.8% as I utilized goroutines and assigned 10 workers per goroutine.

However, when compared to the multi-threaded approach without send-back, latency increased by 4.9% due to the added step, which introduced some overhead.

Zihe's result:

```

/Users/chenzihe/Library/Java/JavaVirtualMachines/corretto-11.0.23/Contents/
Starting load test...
Threads: 30
Albums per thread: 100
Total albums: 3000
Likes/dislikes per album: 2/1
Total requests: 12000

✅ Load test completed!
Total time: 12015 ms
Successful requests: 12000
Failed requests: 0
Total requests: 12000
Throughput: 998.75 req/s
Average response time: 29.10 ms

Process finished with exit code 0

```


Queue album_reviews

Overview

Queued messages last minute ?



Ready 0
Unacked 0
Total 0

Message rates last minute ?



Publish 0.00/s
Deliver (manual ack) 0.00/s
Deliver (auto ack) 0.00/s

Consumer ack 0.00/s
Redelivered 0.00/s
Get (manual ack) 0.00/s

Get (auto ack) 0.00/s
Get (empty) 0.00/s

I used a Java servlet setup with RabbitMQ and MySQL. Under the 10-30-2 configuration, I achieved a throughput of **998.75 req/s** with an average response time of **29.10 ms**, and all **12,000 POST requests** were successful.

While the system handled the load without errors, the response time was higher than others in the group. This may be due to overhead from per-request channel creation or lack of connection pooling. The queue also showed a short spike at the beginning before stabilizing, indicating a temporary lag in message consumption.

Possible Bottlenecks:

Some potential bottlenecks I identified include the overhead of creating a new RabbitMQ channel for every request in the servlet, increased response latency as the number of client threads grows, and possible limitations in MySQL's ability to handle high-concurrency insert operations.

Boyuan Result:

Overview

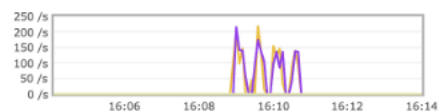
Totals

Queued messages last ten minutes ?



Ready 0
Unacked 0
Total 0

Message rates last ten minutes ?



Publish 0.00/s
Publisher confirm 0.00/s
Deliver (manual ack) 0.00/s

Deliver (auto ack) 0.00/s
Consumer ack 0.00/s
Redelivered 0.00/s

Get (manual ack) 0.00/s
Get (auto ack) 0.00/s
Get (empty) 0.00/s

```
POST Mean Latency: 686.6695081967213
POST Min Latency: 6
POST Max Latency: 20073
POST 50th Percentile: 125.0
POST 99th Percentile: 15065.0

POSTLike Mean Latency: 126.89491803278689
POSTLike Min Latency: 0
POSTLike Max Latency: 1616
POSTLike 50th Percentile: 46.0
POSTLike 99th Percentile: 1245.0

POSTDisLike Mean Latency: 112.6616393442623
POSTDisLike Min Latency: 0
POSTDisLike Max Latency: 15002
POSTDisLike 50th Percentile: 34.0
POSTDisLike 99th Percentile: 1237.0

Throughput: 857 requests/s
Time taken: 105s

Process finished with exit code 0
```

The main issue that Boyuan has experienced is the number of connections that the database can handle. Since POST album info will directly go into the database from the server and the multi-threaded consumer is also connected to the database, there were connection refusals all the time, resulting in a very low successful rate.

3. Conclusion

Despite each member using slightly different tech stacks or configurations, we all converged on the same pattern: asynchronous messaging with RabbitMQ offers significant scalability, especially when consumer threads are tuned properly. Performance bottlenecks often occur at database writes or when the number of consumers fails to match the load.